

 BLOCK 1 – DSA: Queues — Async Task Queue

 Concept

- Sync : request 1 → waits → request 2 starts
 - Async : request 1 → waiting on API → request 2 runs simultaneously 
-

 Queue Model

```
import asyncio
```

```
async def worker(queue):
```

```
    while True:
```

```
        task = await queue.get() # wait for task
```

```
        await process(task)
```

```
        queue.task_done()
```

```
async def main():
```

```
    q = asyncio.Queue()
```

```
    await q.put({"user": "bobby", "msg": "what is array"})
```

```
    await q.put({"user": "anon", "msg": "2**10"})
```

```
    worker_task = asyncio.create_task(worker(q))
```

```
    await q.join()
```

 Think

2 users hit /chat at same time:

Sync → user2 waits ❌

Async → both run 

👉 BUT: ⚠️ They may share same ChatMemory → bug ❌

 LeetCode

 #1700 — Number of Students Unable to Eat Lunch

 Hint

Queue simulation:

- students queue
- sandwich stack

Track whether front element is still possible

 BLOCK 2 – AI Concept: Async Ollama using httpx

 Problem

ollama.chat() = blocking ❌

 Solution

Use:

httpx.AsyncClient 

 Code

```
import httpx
```

```
async def async_ollama_chat(model, messages):
```

```
    async with httpx.AsyncClient(timeout=60) as client:
```

```
        response = await client.post( "http://localhost:11434/api/chat",
```

```
            json={"model": model, "messages": messages, "stream": False})
```

```
        return response.json()["message"]["content"]
```

✅ FastAPI usage

```
@app.post("/chat")
```

```
async def chat_endpoint(req: ChatRequest):
```

```
    memory = store.get_or_create(req.session_id)
```

```
    reply = await async_ollama_chat(
```

```
        memory.model,
```

```
        list(memory.chat_history) + [{"role": "user", "content": req.message}])
```

```
    return {"reply": reply}
```

🔧 BLOCK 3 – Build: Replace Sync with Async

✅ Idea Flow

FastAPI async endpoint

↓

await async_ollama_chat()

↓

httpx call → non-blocking

↓

other users can proceed simultaneously ✅

✅ Tasks

1. Save as week6_day2.py

2. Install:

pip install httpx

3. Write async function

4. Replace ollama.chat()

5. Test multiple requests

-
- ✓ Verify
 - ✓ Multiple requests don't block
 - ✓ Response same as before
 - ✓ Timeout prevents hanging
 - ✓ Server handles parallel calls
-

BLOCK 4 – Reflect

1. Race condition? Multiple users modifying same memory ✗

2. Does async help? If ollama is bottleneck → limited gain ⚠

3. Threading vs asyncio?

I/O → asyncio ✓

CPU → threading/process ✓
